



B-Tree Introduction

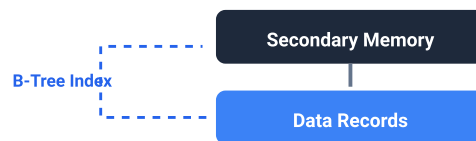
Gate Smashers Revision Notes & Exam Quick Reference

1. What is a B-Tree?

A **B-Tree** is a self-balancing tree data structure designed specifically for primary/secondary memory management and commonly used for **multilevel indexing**.

Dynamic Multilevel Index

Instead of repeatedly creating static indexes of indexes, a B-Tree dynamically organizes and maintains the index using a multi-way search tree structure.



2. Why Do We Need B-Trees?

When searching data stored in secondary memory, multi-level static indexes face performance degradation as levels increase:

✗ Standard Index Issues

- Searching becomes overly complex across multiple file passes.
- Insertion requires multi-level updates.
- Deletion causes massive re-indexing overhead.

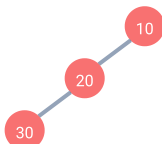
✓ B-Tree Solution

- Maintains dynamic balance automatically.
- Keeps search, insert, and delete log time.
- Minimizes secondary storage disk I/O operations.

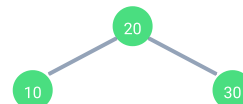
3. Why a Balanced Tree?

In standard trees, skewed structures degrade lookup efficiency to linear scans. B-Trees enforce that **all leaf nodes remain strictly at the same level**.

✗ Unbalanced (Chain)



✓ Balanced B-Tree



4. Basic Tree Terminology

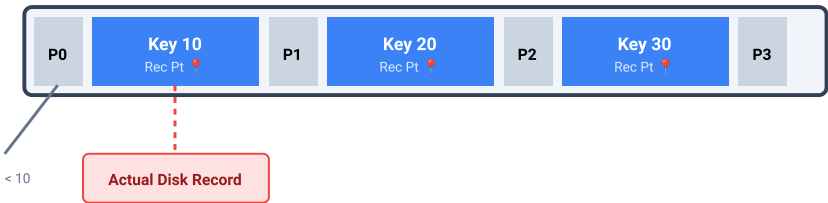
- **Root Node:** The topmost node of the tree hierarchy.
- **Intermediate/Internal Node:** Non-leaf nodes between the root and bottom level.

5. B-Tree Node Anatomy & Components

Unlike standard binary tree nodes that store a single key, a B-Tree node holds **multiple sorted keys** and pointers, allowing large fan-out rates.

Three Essential Node Components

- 1. **Key** (🔑): Search criterion (e.g., Roll No. = 10). Stored in strict **sorted order** inside the node.
- 2. **Record Pointer** (📍): Points directly to the full data record residing in secondary memory.
- 3. **Block / Tree Pointer** (🌳): Points to child B-Tree nodes for range navigation.



Number of Record Pointers
= Number of Keys

Number of Block Pointers
= Number of Children

6. B-Tree Order & Mathematical Bounds

Let **P** represent the *Order of a B-Tree* (Maximum children a node can have):

Property Parameter	Mathematical Formula	Order P = 4 Example
Maximum Children	P	4 Children
Maximum Keys	P - 1	3 Keys
Maximum Record Pointers	P - 1	3 Pointers
Minimum Children (Internal Node)	$\lceil P / 2 \rceil$	$\lceil 4 / 2 \rceil = 2$ Children
Minimum Children (Root Node)	2 (if non-leaf)	2 Children

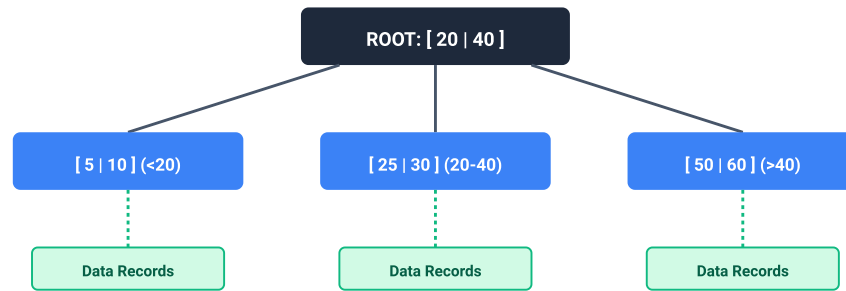
🔍 Block Pointer vs. Record Pointer

Block/Tree Pointer: Points to another B-Tree node to maintain hierarchy.

Record Pointer: Points to the data payload/record in disk storage.

7. Full B-Tree Structural Architecture

Example of a multi-level B-Tree search structure showing routing across ranges:



🔥 Most Important GATE / UGC-NET Exam Points

- **Dynamic Multilevel Index:** B-Tree balances index height automatically.
- **Order P:** Represents the max children capacity of any single node.
- **Max Keys & Record Pointers:** Always equal to $P - 1$.
- **Max Block Pointers:** Always equal to P .
- **Internal Node Minimum Children:** Equal to $\lceil P / 2 \rceil$.
- **Root Minimum Children:** Exactly 2 (when not a leaf).
- **Key Order:** Keys inside every node are strictly sorted.
- **Leaf Height:** All leaf nodes reside strictly at the same level.

🧠 Ultimate Memory Trick

Order = P

↳ Max Children = P

↳ Max Keys = $P - 1$

↳ Max Record Pointers = $P - 1$

Internal Min Children = $\lceil P / 2 \rceil$ | Root Min Children = 2